Dipartimento di Elettronica e Informazione

Politecnico di Milano
prof. ***Carlo Ghezzi***
prof. ***Angelo Morzenti***
prof. ***Pierluigi San Pietro***

20133 Milano (Italia)
Piazza Leonardo da Vinci, 32
Tel. (39) 02-2399.3400
Fax (39) 02-2399.3411

Recupero di Ingegneria del Software – 17 Settembre 2002

Cognome

Nome

Matricola

Sezione (segnarne una) Ghezzi ☐, Morzenti ☐, SanPietro ☐

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno presi in considerazione.
3. È possibile consultare liberamente libri, manuali o appunti, e scrivere a matita. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h30.
6. Punteggio totale a disposizione: 26 punti. La prova è sufficiente ottenendo almeno 16 punti.

Esercizio 1 (12 punti)

Le seguenti classi Punto, Segmento, Triangolo, TriangoloIsoscele rappresentano le note entità geometriche in uno spazio euclideo bidimensionale con coordinate cartesiane.

```
public class Punto {
    // Punto è immutabile
    private float x, y;
    public Punto(float x, float y) {this.x = x; this.y = y; }
    public boolean equals (Punto p) {
        return (p.x ==x && p.y ==y);
    }
    public float distanza (Punto p) {
        return (float)Math.sqrt((p.x-x) * (p.x-x) +(p.y-y) * (p.y-y));
    }
}
```

Della classe Segmento si riporta qui solo la specifica (da non implementare):

```
public class Segmento {
//tipo immutabile, è una coppia ordinata di punti distinti .
    public Segmento(Punto inizio, Punto fine) {
//REQUIRES: inizio e fine sono distinti
// EFFECTS: this come segmento di due punti: (inizio, fine)
    }
    public Punto puntoIniz() {
    //EFFECTS: restituisce l'estremo iniziale di this
    }
    public Punto puntoFin() {
    //EFFECTS: restituisce l'estremo finale di this
    }
    public float lung() {//EFFECTS: restituisce la lunghezza di this
    }
} // fine di Segmento
```

a) Completare negli spazi previsti l'implementazione della classe Triangolo, inclusi AFe RI (supporre come data la definizione della classe TriangoloScorrettoException)**:**

```
public class Triangolo {
    // tipo mutabile, corrispondente a un triangolo NON DEGENERE rappresentato
    // internamente da una terna di segmenti
    //Uno dei tre segmenti è individuato come la base del triangolo
    private Segmento base, lato1, lato2;
    private static boolean verificaTriangolo(Segmento lato0, Segmento lato1,Segmento lato2) {
    //EFFECTS: restituisce true se lato0, lato1, lato2 individuano un triangolo non degenere
    //NB: da non implementare....

    public Triangolo (Punto p0, Punto p1, Punto p2) throws TriangoloScorrettoException {
        // EFFECTS: se p0, p1, p2 individuano un triangolo non degenere, costruisce un triangolo
        // con base(p0,p1), lato1(p1,p2), lato2(p2,p0) altrimenti lancia TriangoloScorrettoException
        this.base = new Segmento(p0, p1); .....
        this.lato1 = new Segmento(p1, p2); .....
        this.lato2 = new Segmento(p2, p0); .....
        if (!verificaTriangolo(base,lato1,lato2)) throw new
TriangoloScorrettoException();.....
        .....
    }
```

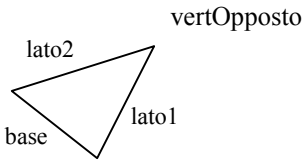
// completare AF

AF(this) = **il triangolo individuato dai tre vertici base.puntoIniz(), base.puntoFinale(), lato1.puntoFinale()**.....

// completare fornendo una descrizione informale (a parole) dell'invariante di rappresentazione
RI(this) =
base != null && lato1!=null && lato2!= null &&
base.puntoIniz ()==lato2.puntoFin () && base.puntoFin ()==lato1.puntoIniz() &&
lato2.puntoIniz() == lato1.puntoFin() && // punto finale di un segm = punto finale segmento successivo
il triangolo individuato dai segmenti
{base, lato1, lato2} è non degenere
.....
.....
.....
.....
// implementare RI con il metodo RepOK() usando i metodi di Punto e Segmento
public boolean RepOK() {
return (
base != null && lato1!=null && lato2!= null &&
base.puntoIniz ()==lato2.puntoFin () && base.puntoFin ()==lato1.puntoIniz() &&
lato2.puntoIniz() == lato1.puntoFin() &&
verificaTriangolo(base, lato1, lato2)

.....
.....
.....
.....
.....
.....
.....
.....

// implementare i seguenti metodi, con riferimento alla figura



}
public Punto vertOpposto () {
//EFFECTS: restituisce il vertice opposto alla base
return lato1.puntoFin();..... .....
}
public Segmento base () { **return base;.**..... .....
}
public Segmento lato1 () { **return lato1;** ....
}
public Segmento lato2 () { **return lato2;**..... .....
}
public void cambiaVertOpposto(Punto p) throws TriangoloScorrettoException {
//EFFECTS: se il triangolo non è degenere
//this.lato1.puntoFin()_post = this.lato2.puntoIniz()_post = p, e this.base_post = base
//altrimenti solleva TriangoloScorrettoException
lato1 = new Segmento(lato1.puntoIniz(),vertOpposto);
lato2 = new Segmento(vertOpposto, lato2.puntoFin());
if (!verificaTriangolo(base, lato1, lato2) throw new TriangoloScorrettoException();
....
....
}
}//fine Triangolo

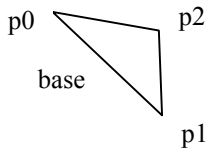
b) Dimostrare che RI è un invariante per Triangolo
Traccia soluzione: il costruttore costruisce base, lato1, lato2 diversi da null e che verificano le altre condizioni dello RI (se non fossero verificate verrebbe lanciata eccezione).
Anche cambiaVertOpposto verifica tutte condizioni dello RI.

c) Si completi inoltre la seguente classe TriangoloIsoscele. Si dica se il principio di sostituzione di Liskov può essere verificato. Si commenti opportunamente la risposta.

public class TriangoloIsoscele extends Triangolo {
// tipo mutabile, corrispondente a un triangolo isoscele NON DEGENERE
//completare, solo se ritenuto necessario, il rep
il rep non va completato.....
// completare AF:
AF(this) =**AF(super)**.....

// completare fornendo una descrizione dell'invariante di rappresentazione
RI(this) = **RI(super) && this.lato1.lung() == this.lato2.lung()**

// implementare RI con il metodo RepOK()
public boolean RepOK() {
return (super.RepOk() && this.lato1().lung() == this.lato2().lung());
//si noti che devono essere chiamati i metodi lato1() e lato2(),
//perché le due variabili omonime sono private nella superclasse
.....
.....
.....
.....
.....
//completare il costruttore:
public TriangoloIsoscele (Punto p0, Punto p1, Punto p2) .....**throws TriangoloScorrettoException**{
// EFFECTS: se p0, p1, p2 individuano un triangolo isoscele non degenere
//(in modo che le lunghezze dei segmenti (p0,p2) e (p1,p2) siano uguali),
// costruisce un triangolo con base (p0,p1), altrimenti lancia TriangoloScorrettoException



super(p0,p1,p2);
if (this.lato1().lung != this.lato2().lung) throw new TriangoloScorrettoException();.....
.....
.....
.....
} //fine costruttore
// Inserire qui la specifica e l'implementazione di altri metodi di Triangolo che si ritiene *necessario* (ri)definire in TriangoloIsoscele
.....
Il princ. di Liskov non vale perche' la cambiaVerticeOpposto deve essere ridefinita in modo da garantire che il triangolo sia isoscele: ha quindi di fatto una precondizione più forte (inclusa nella clausola EFFECTS).

public void cambiaVerticeOpposto(Punto p) throws TriangoloScorrettoException {
//EFFECTS: se this.base.puntoIniz(),this.base.puntoFin() e p individuano
// un triangolo non degenere e ISOSCELE,
//allora this.lato1.puntoFin()_post = this.lato2.puntoIniz()_post = p;
//altrimenti lancia TriangoloScorrettoException
super.cambiaVerticeOpposto(p);
if (this.lato1().lung()!=this.lato2().lung()) throws new TriangoloScorrettoException();
}

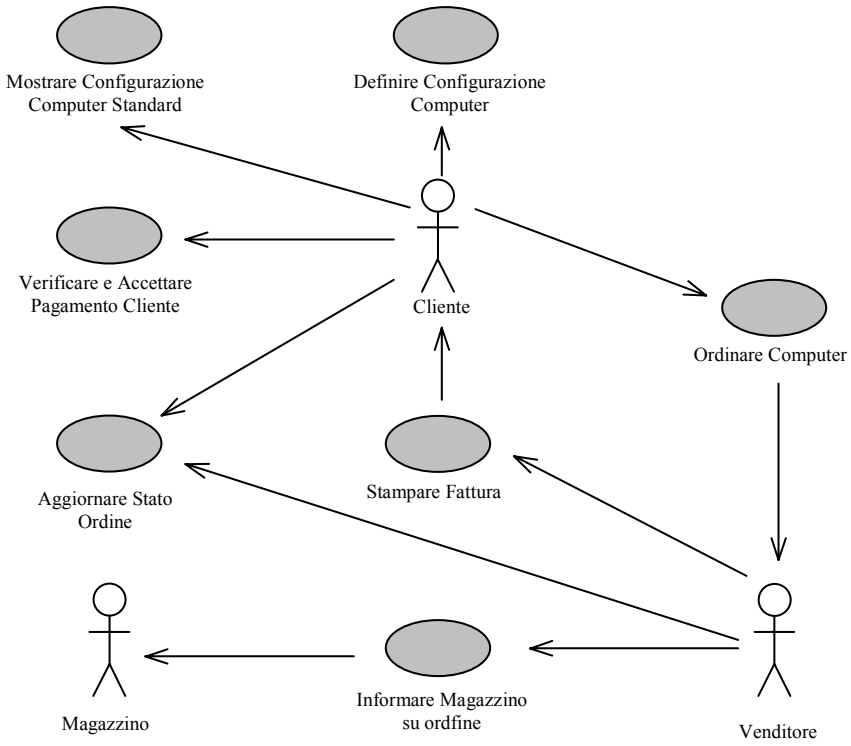
.....
.....
.....
.....
.....
.....
}//fine TriangoloIsoscele

Esercizio 2 (8 punti)

Un produttore di computer offre la possibilità di acquistare computer via internet. Un cliente può selezionare un computer sulla pagina web del produttore. I computer sono classificati in server, desktop e portatili. Il cliente può selezionare una configurazione standard o può definire interattivamente la configurazione desiderata. I componenti configurabili (a esempio le memorie) sono presentati in una lista di opzioni disponibili. Il cliente può ottenere una valutazione del costo di ogni configurazione di computer. Per ordinare, il cliente deve fornire le informazioni per la consegna e il pagamento. Le modalità di pagamento accettate sono: carta di credito e assegno. Una volta che l'ordine è stato inviato, il sistema manda un e-mail di conferma al cliente con i dettagli dell'ordine. Il cliente, mentre attende l'arrivo del computer, può in ogni momento controllare *on line* lo stato dell'ordine. Inoltre il sistema informativo di vendita verifica la solvibilità del cliente, richiede al magazzino la configurazione ordinata, stampa la fattura, richiede al magazzino di spedire il computer al cliente.

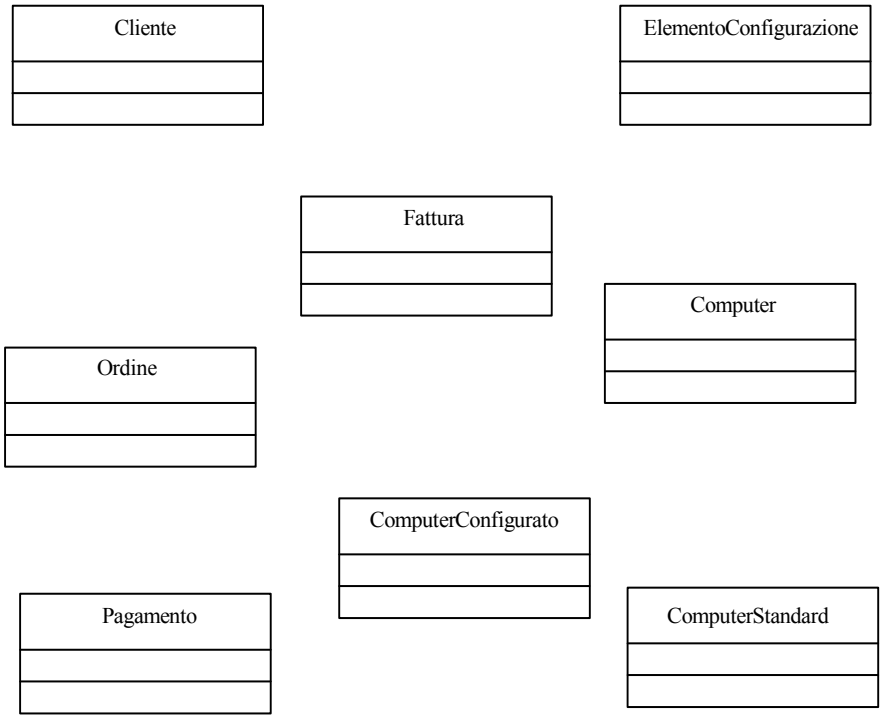
a) Immaginando che nella specifica dei requisiti in UML siano presenti i tre attori: cliente, venditore, magazzino, disegnare i diagrammi di casi d'uso per il sistema sopra descritto.

Sol.

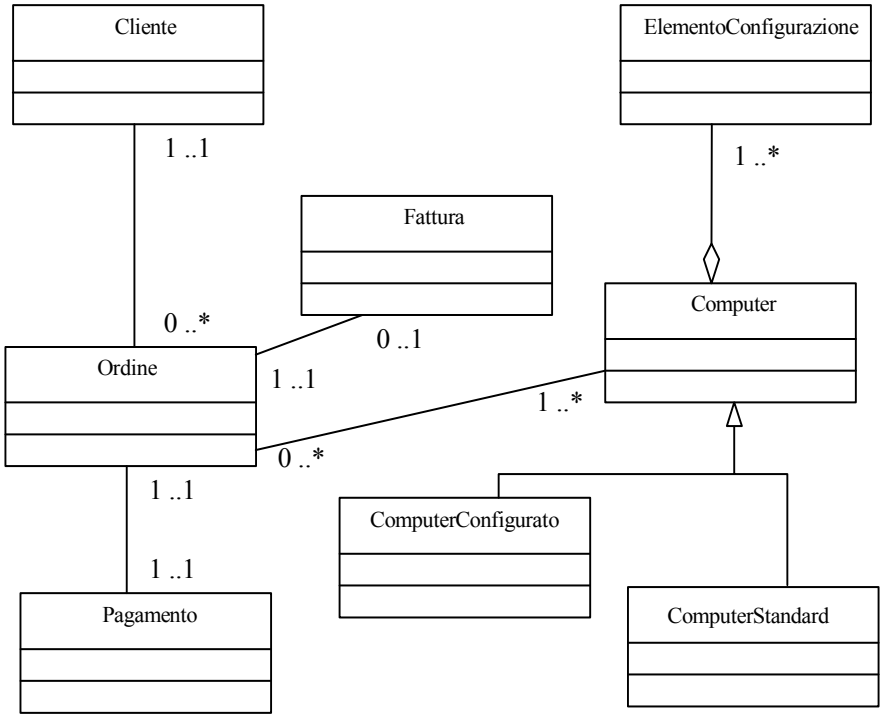


b) Descrivere a parole un possibile scenario significativo e illustrarlo con un sequence diagram.
Sol ovvia.

c) Assumendo di aver individuato le classi mostrate in figura, costruire a partire da essa un diagramma delle classi che includa informazioni relative alle associazioni tra classi (indicando le relative molteplicità), aggregazioni e generalizzazioni.



Sol



Esercizio 3 (6 punti)

1. Si individuino dati di test per il costruttore public TriangoloIsoscele di cui all'Esercizio 1 secondo una strategia di tipo "black box", specificando il motivo per cui ciascun dato viene scelto.
2. Si individuino, per lo stesso costruttore, dati di test di tipo "white box" che coprano tutti i cammini della implementazione data nell'Esercizio 1.

Soluzione

1. **bisogna considerare tutti i modi con i quali si puo' generare una TriangoloScorrettoException: un lato maggiore della somma degli altri due, due angoli coincidenti, tre angoli allineati, i due lati diversi dalla base di lunghezza non uguale; inoltre va considerato i caso di triangolo non degenere e isoscele.**
2. **a un esame superficiale, esistono nell'implementazione due cammini, corrispondenti al caso in cui il triangolo e' isoscele oppure no. Basterebbero quindi per coprire tutti i cammini due casi di test. In realta', bisogna tener presente che la chiamata di super puo' a sua volta generare una eccezione, che provoca l'immediata uscita dal costruttore della sottoclasse con la stessa eccezione. Pertanto in tal caso esistono tre cammini, e quindi 3 casi di test per la copertura secondo un metodo "white box".**